

A Comprehensive Performance Model of Sparse Matrix-Vector Multiplication to Guide Kernel Optimization

Tian Xia[✉], *Member, IEEE*, Gelin Fu, Chenyang Li, Zhongpei Luo, Lucheng Zhang, Ruiyang Chen, Wenzhe Zhao, *Member, IEEE*, Nanning Zheng, *Fellow, IEEE*, and Pengju Ren[✉], *Member, IEEE*

Abstract—Sparse Matrix-Vector Multiplication (SpMV) is important in scientific and industrial applications and remains a well-known challenge for modern CPUs due to high sparsity and irregularity. Many researchers try to improve SpMV performance by designing dedicated data formats and computation patterns. However, out-of-order superscalar CPUs have complex micro-architectures where exist complicated interactions and restrictions among software and hardware factors. It is hard to systematically study the effectiveness of optimization methods on the overall performance, as its benefits may be undermined by other factors. In this paper, we thoroughly study the execution of SpMV on modern CPUs and propose a comprehensive performance model to reveal the critical factors and their relationships. Specifically, we first study the coding characteristics of SpMV kernels to identify key factors worthy of attention. Then we model the execution of SpMV as two overlapped parts: CPU pipeline and memory latency. Both are carefully modeled with related hardware and software factors. We also model SIMD performance with the usage of specific SIMD instructions and vector registers. Experiments show that our model matches the actual execution of real-world processors. Guided by the model, we propose SpV8, a novel SpMV kernel that optimizes critical factors to improve computation efficiency and memory bandwidth. Experiments on Intel/AMD x86 and ARM AArch64 platforms show that SpV8 outperforms several state-of-the-art approaches with large margins, achieving average 3.4× over Intel Math Kernel Library and 1.4× over the best existing approach. Such results indicate that the proposed model is capable of valuable guidance for efficient SpMV optimizations.

Index Terms—Optimization, performance model, sparse-vector matrix multiplication, SIMD

1 INTRODUCTION

THE Sparse Matrix-Vector Multiplication (SpMV) is one of the most commonly-used computation kernels in scientific and industrial domains. Many high-performance scientific tasks such as structural mechanics, fluid dynamics, graph analytics, and circuit simulation rely on SpMV [1]. In these applications, SpMV is the critical performance bottleneck because it is difficult to be efficiently processed by modern CPUs due to the irregular distribution of non-zero values [2]. SpMV can be written as $A \cdot x = y$, where $x \in \mathcal{R}^{n \times 1}$ and $y \in \mathcal{R}^{m \times 1}$ is a dense vector, and $A \in \mathcal{R}^{m \times n}$ is a sparse matrix where non-zeros are in extremely low density (e.g., <0.01%). Compressed Sparse Rows (CSR) is by far the most popular format to efficiently represent a sparse matrix. As depicted in Fig. 1, CSR encodes the

sparse matrix into three arrays: *nnz* as the values of non-zero elements; *colidx* as the column indices of non-zero elements; and *rowptr* that stores the position of each row's first non-zero element in the *nnz* array. For each row, SpMV traverses non-zero elements, uses *colidx* to fetch their corresponding elements in x , and uses multiply-and-add to calculate y .

This process is unfriendly to modern CPU because each row loads from arbitrary positions of x which usually has poor data locality. Moreover, using SIMD usually results in unsatisfactory efficiency, because the irregular distribution of non-zero elements makes it difficult to fully use the parallel processing units [3]. While poor locality and low vector usage are well-known bottlenecks for SpMV [4], we find there are other critical factors that are often overlooked, such as branch prediction and software complexity [5] that greatly influence the execution efficiency of CPU pipeline. All these factors are determined by various software and hardware features including code path, instruction types, pipeline throughput, reorder-buffer, data prefetch, etc. [6] Such complicated interactions among these factors make it difficult to effectively optimize SpMV, because optimizing one or two factors may fail to achieve expected effects as the benefits could be reduced or restricted by other factors.

In this article, we propose a comprehensive performance model of SpMV on superscalar out-of-order CPU, in order to identify the critical factors and shed light on their

- The authors are with the College of Artificial Intelligence, Xi'an Jiaotong University, Xi'an 710049, China. E-mail: {tian_xia, wenzhe, nmzheng, pengjuren}@xjtu.edu.cn, {fugelin, lcy2000, luozhongpei, luchengzhang, ruiyangchen}@stu.xjtu.edu.cn.

Manuscript received 19 April 2022; revised 9 November 2022; accepted 19 November 2022. Date of publication 28 November 2022; date of current version 19 December 2022.

This work was supported in part by National Key Research and Development Program of China under Grant 2021YFB0300900, in part by National Natural Science Foundation of China under Grant 62088102 and in part by Key Research and Development Program of Shaanxi under Grant 2022ZDLGY01-08.

(Corresponding author: Pengju Ren.)

Recommended for acceptance by P. Bangalore.

Digital Object Identifier no. 10.1109/TPDS.2022.3225230

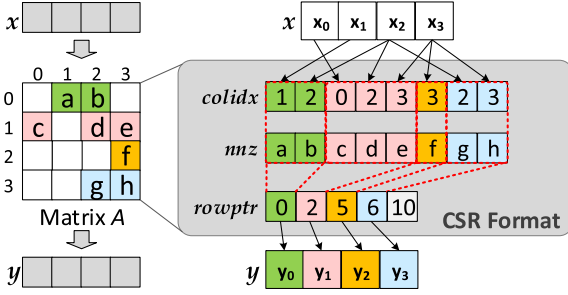


Fig. 1. A typical SpMV process of $A \cdot x = y$, where the sparse matrix A is represented and computed in CSR format.

relationships. Fig. 2 depicts a simplified scheme of our model, which divides the total runtime into computing time (i.e., pipeline execution) and memory overhead (i.e., pipeline stall caused by memory latency), which is a rational paradigm used in micro-architectural performance modelings such as Roofline [7] and ECM [8]. The key factor in our model is *Pipeline Efficiency*, which represents the overall density of valid SpMV operations, i.e., load/store and multiply-and-add of non-zeros. Using this metric, we combine the kernel execution factors of cache miss, vector usage, branch prediction, and software complexity. Multiple hardware factors are included as constraints on both pipeline and memory throughput. For example, the memory-level parallelism (MLP) [9] won't exceed the cache MSHR which is the maximal amount of parallel requests, as depicted in Fig. 2. More importantly, the proposed model reveals some critical principles in SpMV optimization. For example, higher *pipeline efficiency* causes more parallel memory requests to be handled in flight, which is supposed to reduce memory overhead. However, our model shows this is not always beneficial as usual assumptions. As explained in Fig. 2's overlap effect curve, a more efficient pipeline generates more load instructions that throttle the prefetcher's requests making less memory can be hidden. This negative effect only becomes insignificant when MLP gets so high that most

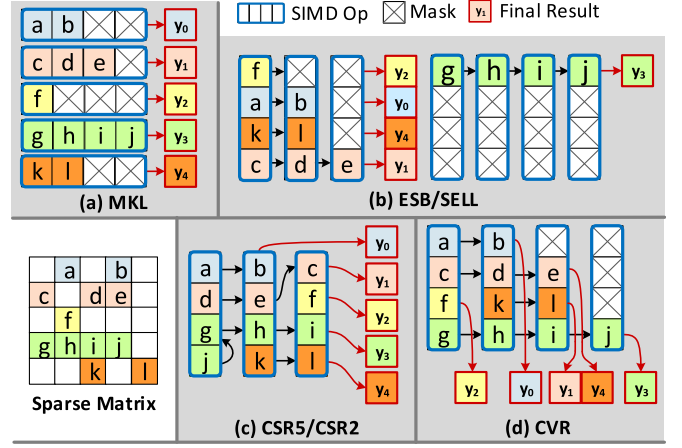


Fig. 3. Examples of some state-of-the-art SIMD-based SpMV kernels with 4-width SIMD vectors: (a) and (b) are less vector-efficient but simpler; (c) and (d) are more vector-efficient and more complex.

memory latency can be amortized. To the best of our knowledge, this is the first work to build such a comprehensive model of SpMV on modern SIMD CPUs.

Furthermore, another contribution in this work is to leverage the proposed model to design SpV8, a novel SpMV kernel that considers *pipeline efficiency* as the target optimization factor to reduce both compute time and memory overhead. Fig. 2 compares SpV8 with some existing kernels, including scalar kernel, MKL [10], CSR5 [11] and CVR [12]. As shown, optimization without theoretical guidance can easily get sub-optimal. For example, MKL and CSR5 have equivalent or even higher memory overheads than scalar kernel though their memory parallelism is higher. On the other hand, guided by our model, SpV8 achieves extremely high *pipeline efficiency* and results in much higher performance by carefully optimizing the combined factors of vector usage, software complexity, and branch prediction. Our evaluations on Intel and ARM CPUs show that SpV8 outperforms other state-of-the-art approaches with significant margins, proving the rationality and value of our model in guiding efficient optimizations. Therefore, we believe the proposed SpMV model and kernel are likely to effectively advance the research of SpMV optimization.

2 BACKGROUNDS

The sparse matrices in SpMV are from various domains and have different distribution features. Many existing studies define two sparse matrix categories [12], [13]: *HPC* where most non-zeros are located in structured bands; and *Scale-Free* where non-zeros positions are highly random and row lengths are very different. Sparse matrices are usually stored in CSR or ELLPACK formats. Unlike CSR in Fig. 1, ELLPACK saves additional zero values to create even-length rows which are suitable for GPU execution. Sliced ELLPACK [14] divides the matrix into row slices in order to reduce the number of fill-in zeros. A more advanced SELL-C- σ [14] format sorts the rows inside each slice to force adjacent rows to have similar non-zero amounts.

SpMV on SIMD architecture is always challenging and has been widely studied in recent years. Fig. 3 depicts several representative state-of-the-art SpMV kernels. Intel

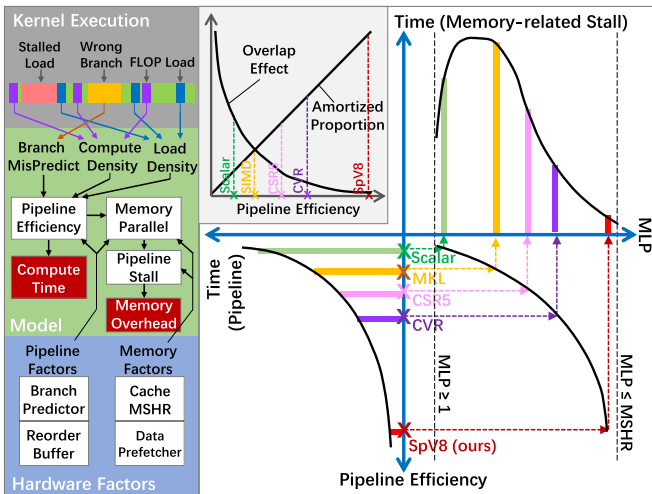


Fig. 2. Simplified scheme of performance model including software and hardware factors based on ARMv8 AARCH64 CPU of GEM5 O3CPU model. The total runtime is divided as compute time (pipeline execution) and memory overhead (pipeline stalled by memory latency). Relationships among pipeline efficiency, prefetch, and MLP amortizing effect are shown. Several kernels are analyzed using *pipeline efficiency* factor.

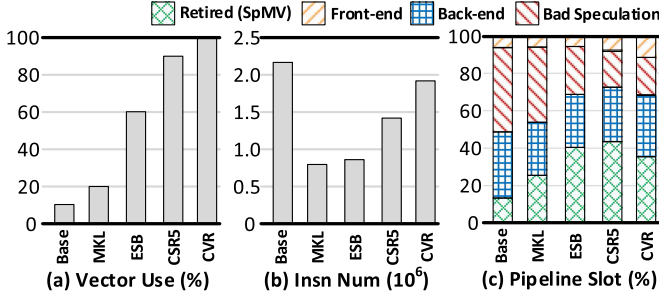


Fig. 4. Evaluation of basic scalar kernel and advanced SIMD-based kernels (MKL, ESB, CSR5, CVR) on matrix *soc-sign-epinions* [18] which includes 131 K rows and 841 K non-zeros. The pipeline slot breakdown is based on Top-Down Micro-architecture Analysis method [19], where pipeline slots are categorized as four types: *Retired* as the useful SpMV execution; *Back-end* as the stalls from memory latency or inadequate function units; *Branch* as branch mispredictions; and *Front-end* as stalls from code fetching and decoding.

Math Kernel Library (MKL) [10] applies a straight-forward method to compute values of the same row with SIMD vectors, which has a simple code path but suffers from low vector usage as shown in Fig. 3a. Some advanced methods including ESB [15] and SELL-C- σ [14] rearrange rows with sorted length and perform SIMD along columns to pursue higher vector usage, as shown in Fig. 3b, which is usually effective but still inadequate on extremely uneven rows such as the *Scale-Free* matrices. Therefore, some methods use more complex patterns to further increase SIMD usage. For example, in order to fully exploit the SIMD width, CSR5 [11] and CSR2 [16] bend long rows to fit into small windows, and CVR [12] feed each SIMD lane with different rows, as shown in Figs. 3c and 3d. However, CSR5 requires flexible adder-tree and partial-sum storage, and CVR needs to deal with the irregular output of y after each vector computation. These extra data structures and more complex computations inevitably undermine the overall computation efficiency.

Algorithm 1. Scalar Sparse Matrix-Vector Multiplication

Require: CSR $A[m][n]$, double-float $x[n]$
Ensure: double-float $y[m]$

```

1: for  $i = 0$  to  $m - 1$  do
2:    $y[i] \leftarrow 0$ 
3:   for  $j = A.rowptr[i]$  to  $A.rowptr[i + 1] - 1$  do
4:      $y[i] \leftarrow y[i] + A.nnz[j] \times x[A.colidx[j]]$ 
5:   end for
6: end for

```

To study the performance of these methods, we evaluate them on Intel CPU as listed in Table 4, and use Intel VTune Profiler [17] to measure the vector usage and pipeline slot breakup of the tested methods, as listed in Fig. 4. As shown in Fig. 4a, MKL has low vector usage as it masks many vector operations, while ESB greatly increases the usage to 60%. Meanwhile, CSR5 and CVR are more advanced to have > 90% vector usage at the cost of more complex computation patterns. Fig. 4b shows that CSR5 and CVR need much more instructions than MKL and ESB even though vectors are better used, meaning that many instructions are used to fulfill the extra complexity. Moreover, Fig. 4c lists the proportion of CPU pipeline slots consumed by various events. As

TABLE 1
Data Structures and Their Access Pattern in SpMV Kernel

Name	Type	Length	Op	Pattern
<i>rowptr</i>	INT32	row number	Read	Stream-in
<i>colidx</i>	INT32	column number	Read	Stream-in
<i>nnz</i>	Float64	<i>nnz</i> number	Read	Stream-in
x	Float64	column number	Read	Random-in
y	Float64	row number	Write	Stream-out

shown, the *Back-end* bottleneck is critical as expected, since SpMV is mostly memory bound. However, we are surprised to see that branch mispredictions cause significant pipeline waste (20%~40%) which is equivalent to or even higher than the *Back-end* bottleneck. This further proves that extra complexity may undermine optimization benefits.

3 KERNEL ANALYSIS

3.1 Code Breakdown

A typical CSR-based SpMV kernel is listed in Algorithm 1, which consists of two loops: the inner loop to accumulate all non-zero elements in each row, and the outer loop to compute all rows. There are five data structures: x as the input vector, y as the output vector and $\{rowptr, nnz, colidx\}$ to represent the matrix. Table 1 lists the types and access patterns of these arrays. Most data is accessed in serial streams, except for the x vector whose load addresses are determined by $colidx$ and are usually random. To analyze the execution of Algorithm 1, Fig. 5 presents its assembly code generated by ARMv8 GCC compiler and highlights memory access (ld/st) instructions, FLOP operations, and branches. As depicted, there are three conditional jump instructions in code path, noted as *Branch-1/2/3*. Their jump directions are based on the following conditions respectively: 1) whether the current row is empty; 2) whether all non-zero elements of the current row are computed; 3) whether all rows are computed. From experience we know that conditions of *Branch-1* and *Branch-3* are usually *false* so that their jump directions are mostly fixed during the execution. However, the direction of *Branch-2* depends on the random length of each row that is unpredictable, making *Branch-2* the most critical branch. Based on direction of *Branch-2*, we breakdown SpMV kernel into two code blocks as shown in Fig. 5: *Block-A* that loads nnz and x to perform floating-point multiply-and-add in each row; *Block-B* that writes back the result y of current row and changes to the next row by loading the next *rowptr*. *Branch-1* and *Branch-3* are included in *Block-B* as their directions are assumed to be fixed. Then SpMV can be considered as the alternate executions of *Block-A* and *Block-B*, while *Block-B* is always followed by *Block-A*. The amount of *Block-A* equals the number of non-zeros and the amount of *Block-B* is the number of matrix rows. It can be noticed that computation instruction (FLOP) is in low density as 1/7 in *Block-A* and 0/11 in *Block-B*.

3.2 Out-of-Order Execution

Modern CPUs use the out-of-order technique to execute multiple instructions in parallel. Fig. 6 depicts a typical out-of-order CPU micro-architecture with hardware features

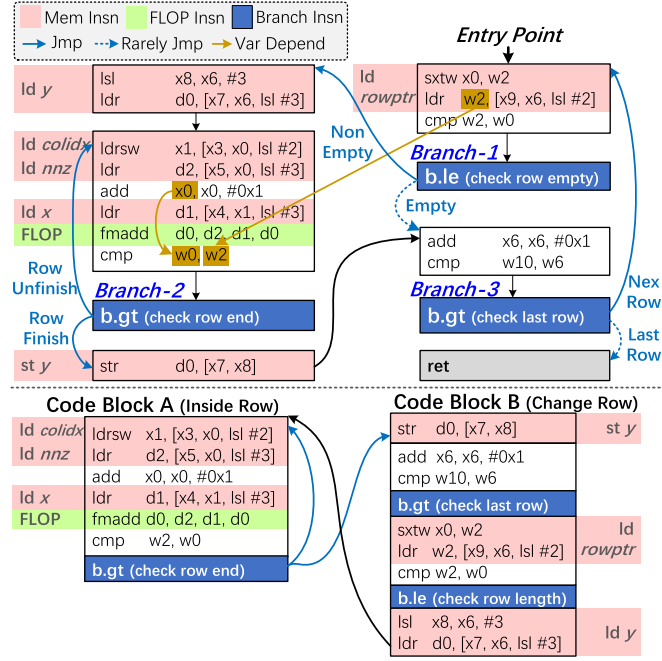


Fig. 5. SpMV kernel code path on ARMv8 instruction-set compiled by GCC -O3 option. Dividing from the *Branch-2*, we break down the codes into two blocks: *Block-A* to compute each row, *Block-B* to change rows.

including branch predictor, register rename, and reorder buffer (ROB). An out-of-order CPU allows to raise multiple data requests to the memory hierarchy simultaneously. L1 and L2 caches are equipped with Miss-Status Holding Registers (MSHR) that handle parallel cache miss requests. To simplify the model, we first ignore factors that have no influence on SpMV performance: 1) The instruction cache (I-Cache) never misses as SpMV kernels have a small code footprint (< 1 KB); 2) The floating-point function unit is not a bottleneck as SpMV is much lower than CPU's peak performance; 3) Register rename won't block CPU pipeline as physical register file is always enough. Meanwhile, there are several factors that deserve further discussion:

3.2.1 Data Prefetch

Data prefetchers capture data access patterns and fetch them in advance to hide long memory latency. There exists a large body of prior research on different kinds of prefetchers. However, commercial processors such as Intel and ARM are limited to conservative prefetcher designs such as next-line and stride prefetchers [20]. Our model considers a stride prefetcher residing between the L1 and L2 cache, as depicted in Fig. 6. Stride prefetcher can capture patterns for addresses separated by a constant stride and is effective for array access patterns. So the stream-in arrays (*rowptr*, *colidx*, *nnz*, *y*) can be easily prefetched, making them usually have low memory latency. In contrast, access on *x* is often not strictly streamed, and sometimes even highly random, making it difficult to be efficiently prefetched [21].

3.2.2 Branch Prediction

Branch prediction learns branch patterns from historical records. When a branch instruction is encountered, the CPU predicts the jump direction and continues to run before the

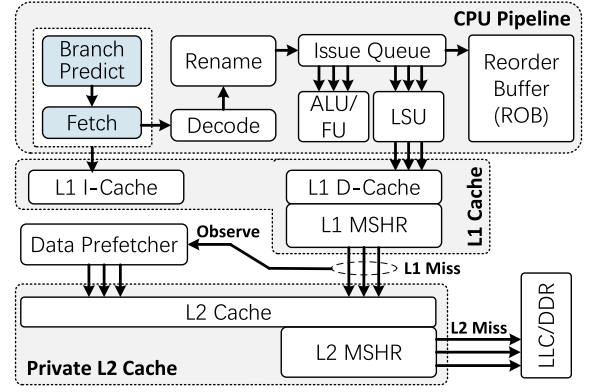


Fig. 6. An example scheme of modern out-of-order super-scalar CPU architecture and memory hierarchy.

branch condition is actually resolved. Misprediction is expensive for high-frequency CPUs with deep pipelines. When a misprediction is detected, CPU has to recover its status of branch point, as well as flush the entire pipeline in order to remove the speculatively issued or executed instructions. The only critical branch of the SpMV kernel is *Branch-2* which depends on row length. Fig. 5 depicts the variables to compute *Branch-2* condition, whose jump direction depends on the comparison result between row length ($rowptr[i+1] - rowptr[i]$) and the iteratively incremented variable i , as listed in Algorithm 1 line-3. In the case of CSR format where adjacent rows have different and random lengths, predictions on this branch are likely to be wrong. Fortunately, because *rowptr* array can be prefetched in a stream-in pattern, the calculation of $rowptr[i+1] - rowptr[i]$ is fast solved. The increment of i is also fast as a local variable. Hence the branch condition result can be quickly obtained, causing a quick misprediction detection so that the CPU won't go far on the wrong code path, which reduces the branch mispredictions penalty. This penalty can be further optimized by putting together rows of similar length to simplify the prediction, which is performed in advanced formats such as SELL-C- σ [14] and our approach.

3.2.3 Long-Latency Memory Access

Long memory latency occurs when a data request misses in the L2 cache and has to access the last-level cache (LLC) or DDR. Technically, we define a data request to be *useful* if its fetched data is actually used by subsequent non-speculative instructions [22]. We assume that in SpMV all the long-latency data requests are *useful*. In other words, there is no invalid LLC/DDR access caused by wrong prefetching or wrongly-executed speculative load instructions. This is because wrong branch predictions are fast detected so that speculative data requests on the wrong code path would be quickly removed and won't get issued to L2 MSHR. Meanwhile, we need to consider the overlapping effect between CPU pipeline and memory accesses, which is significant due to the data prefetching as well as the out-of-order execution mechanism. For example, Intel Skylake CPU cores have a 224-depth ROB size, so that hundreds of instructions can be executed in parallel with the memory access before the pipeline is entirely stalled.

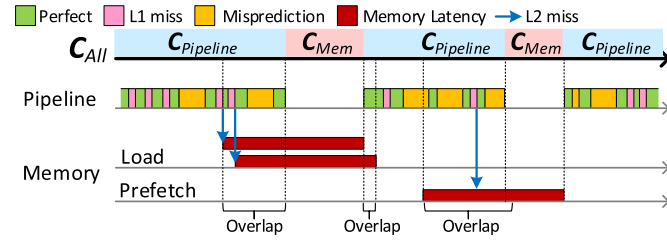


Fig. 7. Break-down of SpMV execution cycles as two overlapped parts: the CPU pipeline and the long memory latency.

3.2.4 SIMD

High-end CPUs use SIMD to improve parallelism, as one SIMD instruction can operate multiple operands by using vector registers. Masks are used to select only a part of vector elements for operation. A naive SIMD-based SpMV kernel has the same code structure of Fig. 5, but uses SIMD load/store and FLOP instructions to operate vectors of variables, except for the *rowptr* which is still loaded one-by-one to control the inner loop. Considering the irregular non-zero distribution in the sparse matrix, SIMD kernels tend to use masks to handle the irregularity, as depicted in Fig. 3. Thus the usage rate of SIMD vectors is a critical factor. The density of SIMD instructions should also be concerned, as SpMV kernels may include extra pre-/post-processing operations which undermine the efficiency of SIMD computations.

4 PERFORMANCE MODEL

In this section, we build a generic performance model of single-core SpMV kernel. Generally, we calculate the single-core performance as:

$$FLOPS = \frac{OpNum}{C_{All}} \times freq, \quad (1)$$

where *OpNum* is the number of SpMV arithmetic operations that depends on the number of non-zero values (*nnz*) in matrix, *freq* is CPU frequency, and *C_{All}* is the cycles of SpMV execution. As depicted in Fig. 7, *C_{All}* can be considered as the combination of two parallel parts: CPU pipeline execution which occurs inside the CPU core, and long-latency memory access caused by cache misses that may stall the entire CPU pipeline, as the following:

$$C_{All} = C_{Pipeline} + C_{Mem} \quad (2)$$

TABLE 2
List of Controlled Experiments of SpMV Kernel Performance Based on GEM5 CPU Model

Variant	Prefetch	Branch Predict	L1 D-Cache	L2 Cache
<i>normal</i>	stride	normal	32 KB	256 KB
<i>no_prefetch</i>	none	normal	32 KB	256 KB
<i>no_spec</i>	stride	all correct	32 KB	256 KB
<i>pipeline</i>	stride	normal	32 KB	∞
<i>perf_w_l1</i>	stride	all correct	32 KB	∞
<i>perfect</i>	stride	all correct	∞	∞

In order to study the effects of software and hardware factors on SpMV, we use a simulated CPU based on GEM5 [23] which is a standard ARMv8 4-way superscalar out-of-order core with private 32 KB L1 and 256 KB L2 caches, 224-depth reorder buffer, and 16/32-depth L1/L2 cache MSHR. Based on this CPU, we create a group of controlled experiments in Table 2, which allows us to manipulate some factors and observe the outcomes. For example, to study the effect of prefetch, we turn off the prefetcher in *no_prefetch* and compare its performance with *normal*. Similarly, we eliminate the speculation penalty by forcibly setting the branch prediction results to be correct, and avoid the memory penalty by using infinite L1/L2 cache size (512 MB in experiments). The tested sparse matrices in Table 3 include both *HPC* and *Scale-Free* categories. The influences of some factors are separately discussed for the two types.

4.1 CPU Pipeline Cycles

4.1.1 Perfect Runtime

We first estimate the pipeline execution cycles (*C_{Pipeline}*) as:

$$C_{Pipeline} = C_{Perfect} + C_{L1Miss} + C_{BpMiss} \quad (3)$$

which includes ideal runtime (*C_{Perfect}*), L1 cache miss overhead (*C_{L1Miss}*) and speculation penalty (*C_{BpMiss}*). Perfect execution assumes that the CPU is always on the right code path and all the required data can be found in the L1 cache, so that the execution cycles are only influenced by the CPU micro-architecture, such as front-end pipeline throughput (i.e., fetch, decode, rename, dispatch), the available function units and the commit width. In this case, perfect runtime (*C_{Perfect}*) can be calculated as:

$$C_{Perfect} = \frac{InsnNum}{IPC} \quad (4)$$

TABLE 3
List of Tested Matrices in *HPC* and *Scale-Free* Types Selected From the SuiteSparse Collection Benchmark

<i>HPC</i>	row & nnz	<i>Scale-Free</i>	row & nnz
<i>ut2010</i> ^{††}	115406,572066	<i>soc-sign-epinion</i> ^{††}	131828,841372
<i>internet</i> ^{††}	124651,207214	<i>sx-superuser</i> ^{††}	194085,924886
<i>email-EuAll</i> ^{††}	265214,420045	<i>web-Stanford</i> ^{††}	281903,2312497
<i>IMDB</i> [†]	428440,3782463	<i>com-Amazon</i> [†]	334863,1851744
<i>RoadNet-PA</i> ^{††}	1090920,3083796	<i>web-Google</i> ^{††}	916428,5105039
<i>web-BerkStan</i> [†]	685230,7600595	<i>wiki-Talk</i> [†]	1000005,3105539
<i>as-Skitter</i> [†]	1696415,22190596	<i>com-Youtube</i> [†]	1134890,5975248

[†]: studied on GEM5 simulator.

^{††}: studied on Intel XEON Skylake CPU.

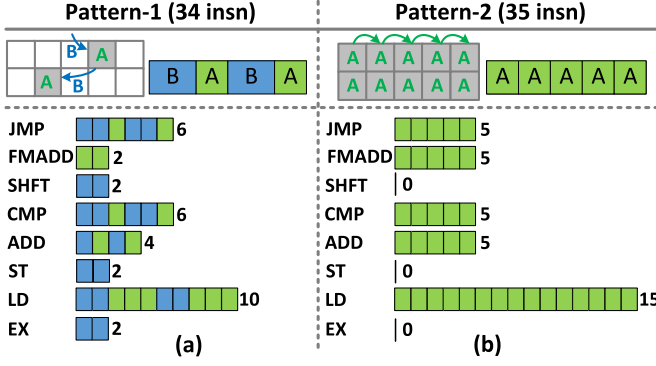


Fig. 8. Two extreme execution patterns of SpMV kernel and their issued instructions in a window of 34~35 instructions: (a) alternate *Block-B* and *Block-A* in short rows; (b) repeated *Block-A* in long rows.

where IPC^* is the theoretical perfect Instruction-Per-Cycle under such ideal execution, and $InsnNum$ is the total instruction number. As depicted in Fig. 5, SpMV is broken down as *Block-A* and *Block-B*, whose amounts equal to the number of non-zeros (nnz) and the number of rows ($RowNum$) respectively. So the total instruction number equals to:

$$InsnNum = nnz \times BlockSize_A + RowNum \times BlockSize_B \quad (5)$$

where $BlockSize_A=7$ and $BlockSize_B=10$ are the instruction numbers of *Block-A* and *Block-B*.

Meanwhile, optimal IPC^* depends on the potential parallelism of instructions, which is influenced by the pattern of codes. Fig. 8 presents two extreme patterns: 1) for very short rows, $\{Block-B, Block-A\}$ is repeated as Fig. 8a; 2) for a very long row, *Block-A* is repeated for many times as Fig. 8b. For both patterns, the amounts of different instruction types issued during the same execution window (34~35 instructions) are depicted. As shown, *Pattern-2* imposes high pressure on load/store unit (LSU), while *Pattern-1*'s instructions are more spread over multiple function units. So *Pattern-1* has higher IPC^* as its instructions can be more efficiently issued and executed in parallel. According to Fig. 8, it takes at least 10 cycles to issue all 34 instructions of *Pattern-1*, so we estimate optimal $IPC_{P1}^*=3.4$. Similarly, we estimate *Pattern-2*'s IPC_{P2}^* to be $34/15=2.2$. Then overall IPC^* can be estimated using the proportion of different patterns:

$$IPC^* = \epsilon \times IPC_{P1}^* + (1 - \epsilon) \times IPC_{P2}^* \quad (6)$$

$$\epsilon = \frac{InsnNum_{P1}}{InsnNum} = \frac{RowNum \times (BlockSize_A + BlockSize_B)}{InsnNum} \quad (7)$$

where ϵ is the proportion of *Pattern-1* in total instructions.

4.1.2 L1 Cache Miss Penalty

Load/store operations that miss in L1 cache and hit in L2 cache are considered as part of the CPU pipeline cycles as they are fast resolved inside the CPU core. Since LSU is the bottleneck as shown in Fig. 8, such L1 cache misses would reduce the throughput of LSU, which effect can be modeled as extra latency add-on to load instructions [24]. Generally, the total extra cycles caused by L1 cache misses are:

Authorized licensed use limited to: Xian Jiaotong University. Downloaded on May 31, 2023 at 02:59:05 UTC from IEEE Xplore. Restrictions apply.

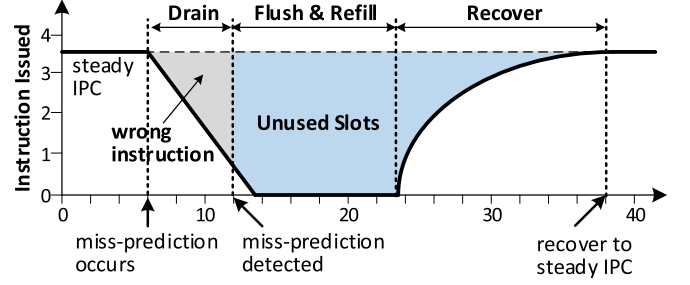


Fig. 9. Branch misprediction penalty is measured as the wasted slots during the three stages of *Drain*, *Flush&Refill* and *Recover*, which is the shadowed area above the curve.

$$C_{L1Miss} = L1MissNum \times \Delta_{L1} \quad (8)$$

where $L1MissNum$ is the amount of L1 cache miss that hit in the L2 cache, and Δ_{L1} is the average L1 cache miss penalty. In our model, it takes 1 cycle to transfer one cache line between L2 and L1 cache. According to experiments, the value of Δ_{L1} is measured to be 0.9.

4.1.3 Speculation Penalty

A branch misprediction is detected when branch condition is resolved at the execution stage, which triggers CPU to flush the entire front-end pipeline and remove the wrong instructions that have already entered issue queues, functional units, and ROB. Fig. 9 shows the influence of branch misprediction by depicting the curve of valid pipeline throughput, i.e., the number of correct instructions issued per cycle. According to prior research [24], the curve has three stages: (1) Error occurs but is not detected so pipeline throughput remains steady, but the valid throughput drains as more miss-speculative instructions enter the pipeline. (2) After error detection, all speculative instructions are removed from the pipeline so that the throughput quickly drops to zero until new instructions on the correct branch are fetched and refill the pipeline. (3) Eventually, the throughput slowly recovers to its steady point with a logarithmic curve. The penalty of each misprediction is measured as the wasted pipeline slots during the above stages as depicted in Fig. 9b, which is usually significantly greater ($1.5 \times - 3 \times$) than the front-end pipeline [24] [25]. *Pattern-1* has higher penalty than *Pattern-2* as it has higher pipeline throughput which causes faster *Drain* and longer *Recover* period. Our experiments measures their penalties to be $\Delta_{BP}^{P1} = 20$ cycles and $\Delta_{BP}^{P2} = 11$ cycles respectively, which is within the rational range considering the CPU's front-end pipeline is 7. Then, we estimate the average penalty of each misprediction as:

$$\Delta_{BP} = \epsilon \times \Delta_{BP}^{P1} + (1 - \epsilon) \times \Delta_{BP}^{P2} \quad (9)$$

The total speculation penalty cycles can be calculated as:

$$C_{BPMiss} = BPMissNum \times \Delta_{BP} \quad (10)$$

4.2 Memory Latency

As depicted in Fig. 7, L2 cache miss triggers long latency that may cause the CPU pipeline to stall. The total number of stalled CPU cycles can be calculated as:

$$C_{Mem} = (1 - Overlap) \times C_{L2Miss} \quad (11)$$

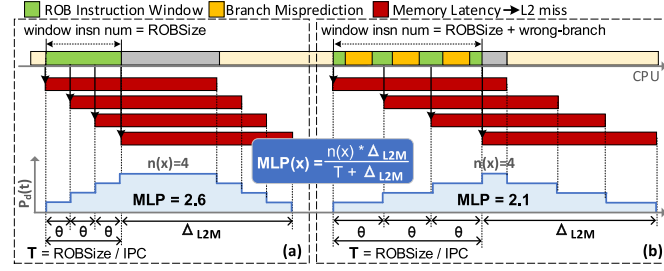


Fig. 10. Examples of MLP calculation with different L2 cache miss intervals (θ) cases with and without branch mispredictions.

where C_{L2Miss} is the total cycles of L2 cache miss latency, and $Overlap$ describes the parallelism between pipeline and memory access. Then we calculate the value of C_{L2Miss} as:

$$C_{L2Miss} = \frac{L2ReqNum \times \Delta_{L2M}}{MLP} \quad (12)$$

where $L2ReqNum$ is the total number of L2 cache misses,¹ Δ_{L2M} is the average cycles to load a cache line from LLC/DDR which is set at 200 in GEM5 simulator, and MLP measures the memory-level parallelism that allows memory latency to be amortized. For simplicity, we rewrite Eq. (11) as:

$$C_{Mem} = \lambda \times \Delta_{L2M} \times L2ReqNum, \lambda = \frac{1 - Overlap}{MLP} \quad (13)$$

where λ is *impact factor*. In our model, λ is important for data transfer analysis as it combines two critical effects: the core-memory overlapping and the parallel memory amortizing, both of which greatly influence the data transfer overheads. This section mainly studies the modeling of λ factor.

4.2.1 Memory Level Parallelism

MLP reflects the average number of parallel memory access, which is formally defined in [22]. We define the number of outstanding L2 cache MSHR requests at given moment t is $P(t)$, and MLP is defined as the expectation of $P(t)$ for all cycles when $P(t) \geq 1$, which formally equals to:

$$MLP = E(P(t) | P(t) \geq 1) = E\left(\mathcal{B}_{L2}(\mathcal{B}_{L1}(P_d(t)) + P_p(t))\right) \quad (14)$$

where $P_d(t)$ and $P_p(t)$ represent the L2 cache miss number generated by core and by prefetcher, respectively. $\mathcal{B}_{L1}$ and $\mathcal{B}_{L2}$ stand for the restriction of L1/L2 cache MSHR, which limit the maximal number of outstanding memory requests.

We first discuss the optimistic case where L1/L2 cache MSHRs are large and all stream-in data are prefetched, so that $P_d(t)$ only includes loads on x . We can rewrite Eq. (14) as:

$$MLP = E(P_d(t)) + E(P_p(t)) = MLP(x) + MLP(pf) \quad (15)$$

In order to estimate $MLP(x)$, we study the parallelism within the instruction window of reorder buffer (ROB) [26]. We define the probability of one load x instruction causing

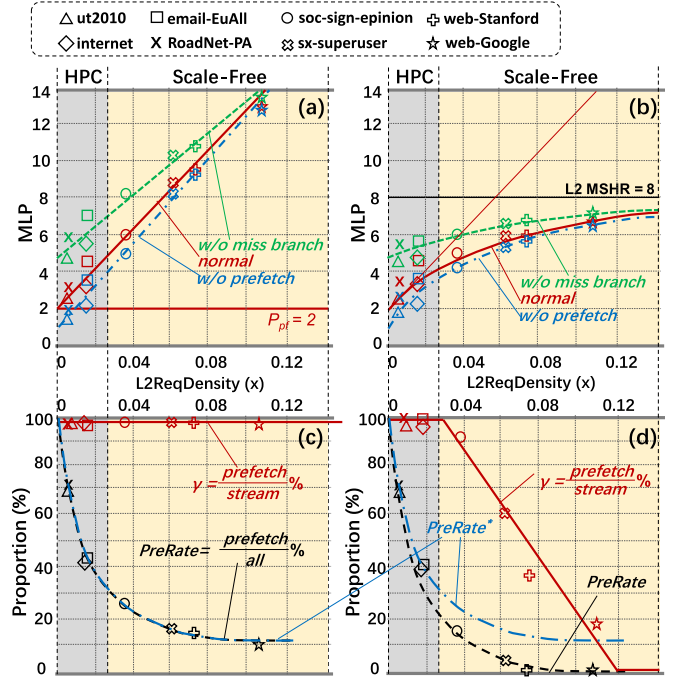


Fig. 11. (a)(b) MLP - $L2ReqDensity(x)$ relationship which is influenced by the MSHR bound, prefetching and branch mispredictions. (c)(d) Prefetching effect factors of $PreRate$ and γ with their relationship with $L2ReqDensity(x)$ when MSHR is the bound or not.

L2 cache miss to be $L2ReqDensity(x) = \frac{L2ReqNum(x)}{InstNum}$, so the L2 cache miss number to occur in the ROB window equals to:

$$n(x) = ROBSize \times L2ReqDensity(x) \quad (16)$$

As depicted in Fig. 10a, we assume the $n(x)$ loads occur evenly during the window with intervals of θ cycles, and the sum of all the intervals equals the execution cycles of ROB window instructions, which is formulated as:

$$T(n(x), \theta) = \theta \times (n(x) - 1) = \frac{ROBSize}{IPC_{pl}} \quad (17)$$

where IPC_{pl} is the CPU pipeline IPC obtained from Eq. (3). Then, as shown in Fig. 10a, we can calculate $MLP(x)$ to be:

$$MLP(x) = \frac{n(x) \times \Delta_{L2M}}{T + \Delta_{L2M}} = \frac{n(x) \times \Delta_{L2M}}{\frac{ROBSize}{IPC_{pl}} + \Delta_{L2M}} \quad (18)$$

We can derive that by improving pipeline IPC , e.g., reducing branch mispredictions, $MLP(x)$ can be optimized, as shown in Fig. 10b. Meanwhile, with empirical estimation of $MLP(pf) = 2$, we can create the MLP model using Eq. (15). Fig. 11a reports the tested MLP values of benchmarks, which are in good match with our model. We also test without prefetch (*no_prefetch*) and without branch mispredictions (*no_spec*), the results prove that prefetching and better branch prediction can effectively improve MLP .

Furthermore, considering cache bandwidth is likely to be the bottleneck when SpMV is executed by multiple cores, we set the L2 cache MSHR size to be 8 in order to study the pessimistic case of cache bandwidth saturation. As depicted in Fig. 11b, we witness MLP increases with $L2ReqDensity(x)$ but gradually slows down as it's bounded by L2 MSHR, as

1. Several L2 cache misses that belong to the same cache line are merged as one L2 cache MSHR request, which is counted as one L2 cache miss in our model.

suggested in Eq. (14). This situation is considered as the lower bound of our model and will be studied later.

4.2.2 Overlap Factor

Memory latency is overlapped for two reasons: the out-of-order mechanism that allows successive instructions to execution when a ld/st instruction misses in the L2 cache, and the prefetcher that automatically fetches data in parallel with the CPU pipeline. We model the overlap factor as:

$$Overlap = \alpha + \gamma \times PreRate \quad (19)$$

where α represents the intrinsic overlapping due to out-of-order execution, and the second part measures the prefetching effect. To estimate α , consider Fig. 10 where each L2 cache miss latency is overlapped with the θ cycles of out-of-order execution. With the assumption that L2 cache misses are evenly distributed, we can estimate α as:

$$\alpha = \frac{\theta}{\Delta_{L2M}} = \frac{1}{L2ReqDensity(x)} \times \frac{1}{IPC_{pl}} \times \frac{1}{\Delta_{L1M}} \quad (20)$$

However, compared with the out-of-order mechanism, prefetch is usually more effective to hide memory latency. We define $PreRate = \frac{L2ReqNum(Prefetch)}{L2ReqNum}$ to measure the proportion of prefetched data. In the optimistic case, the total prefetched data equals the size of $\{mnz, colidx, rowptr, y\}$, and their L2 cache line number can be calculated as:

$$L2ReqNum(stream) = p \times InsnNum \quad (21)$$

where p is the average stream-in cache line number used by an instruction. Then the ideal $PreRate^*$ can be calculated as:

$$\begin{aligned} PreRate^* &= \frac{p \times InsnNum}{p \times InsnNum + L2ReqNum(x)} \\ &= \frac{1}{1 + \frac{L2ReqDensity(x)}{p}} \end{aligned} \quad (22)$$

Consider the code blocks defined in Fig. 5, we can calculate that for *Block-A* $p = 0.023$ and for *Block-B* $p = 0.018$. For simplicity, we approximately set $p = 0.02$ in our model. As shown in Fig. 11c, the tested matrices match with Eq. (22) perfectly.

We use factor γ to measure the prefetch quality including the timeliness and coverage rate of stream-in data. Ideally we assume $\gamma = 1$. However, as prefetcher and pipeline share the cache MSHR slots when cache bandwidth is saturated, γ could decrease. Fig. 5d depicts a pessimistic case where L2 cache bandwidth is the bottleneck. We witness that $\gamma = 1$ at the beginning, meaning that they can fetch most stream-in data of *HPC* matrices. On the other hand, large *scale-free* matrices generate many L2 cache misses that occupy the L2 cache MSHR resource, which throttles prefetcher requests. As a result, only a small part of stream-in data ($\leq 50\%$) is prefetched, causing γ to fast drop.

As both parts of Eq. (19) declines with more random x access, it can be observed in Figs. 10c and 10d that memory latency is effectively hidden for *HPC* matrices due to prefetch, but much less overlapped in *scale-free* matrices.

4.2.3 Impact Factor

As defined in Eq. (13), λ is the *impact factor* to measure effective penalty of each L2 cache miss after being overlapped and

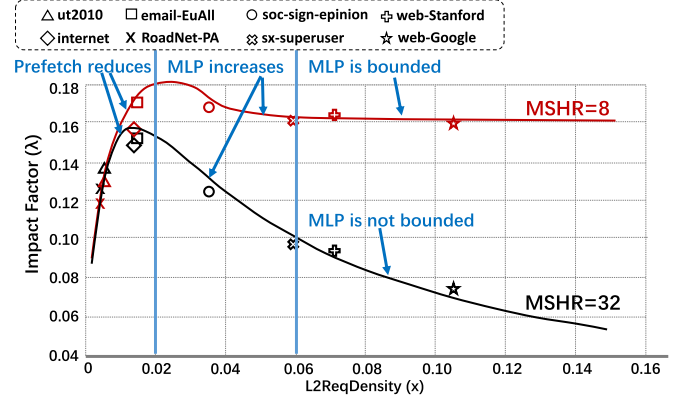


Fig. 12. The model of λ - $L2ReqDensity(x)$ curves with optimistic case ($L2$ MSHR = 32) and pessimistic case ($LSHR = 8$). The rise-fall pattern is due to the declining prefetching effect and the improving *MLP* which brings better memory latency amortization. When *MLP* gets bounded by MSHR, the value λ stops reducing.

amortized. Considering SpMV is usually memory-bounded, λ is a critical factor as it determines how much each L2 cache miss contributes to the total runtime. Therefore reducing λ is critical for SpMV optimization. We combine *MLP* and *Overlap* models to estimate λ with consideration of two extreme cases: the optimistic with sufficient cache bandwidth and the pessimistic where MSHR is the bottleneck. The optimistic model can be analytically built using Eqs. (18)–(19). Meanwhile, the pessimistic model is approximated using data listed in Figs. 11b and 11d, as the contention effect on cache bandwidth is difficult to be analytically described. Fig. 12 depicts the modeled λ curves, which are in good match with experiment results. We witness a rise-fall pattern in both curves, which surprisingly indicates that higher x cache miss density doesn't always lead to lower λ . For *HPC* matrices, λ is dominated by *Overlap*, so higher $L2ReqDensity(x)$ may deteriorate the memory penalty instead of alleviating it as the prefetching effect gets lower, especially when cache bandwidth is bottleneck. Meanwhile, for *scale-free* matrices where *Overlap* is less significant, λ is dominated by *MLP* so that higher *MLP* is beneficial to amortize memory penalty. We also observe λ stops improving when *MLP* reaches the MSHR bound, which may occur in multi-thread kernels.

4.2.4 Model Evaluation

Our model studies pipeline performance and data transfer with different methodologies. $C_{pipeline}$ is analytically modeled, and C_{Mem} is partly analytical by using the observed numbers of L1/L2 cache miss because the random accesses on cache hierarchy are difficult to predict. Our model also includes some critical micro-architectural factors such as the sizes of L1/L2 cache MSHR and reorder-buffer, and the latency to load a cache line from LLC or DDR.

To evaluate our model, we use it to estimate the pipeline $C_{pipeline}$ and overall cycles C_{All} of scalar SpMV kernel using the Table 3 matrices and compare it with GEM5 simulator results. As depicted in Fig. 13a, our model estimations match well with GEM5 on all matrices for both $C_{pipeline}$ and C_{All} . Moreover, we test our model with a real-world CPU, i.e., an Intel Xeon Gold 6146 CPU with Skylake-X architecture. Large matrices of up to 25 million non-zeros are tested. To measure the real-world pipeline execution time, we modified the *colidx* to restrict its range within 64 so that x

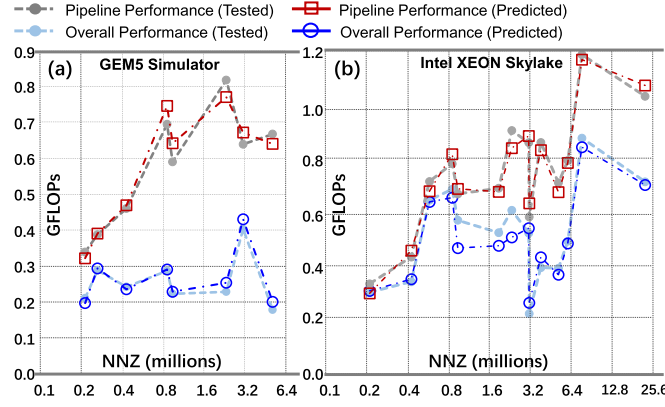


Fig. 13. Comparison of our model with GEM5 simulation results and real-world Intel XEON CPU results on pipeline execution ($C_{pipeline}$) and overall (C_{All}) single-core performances using Table 3 matrices.

loads always hit in the L1 cache, making the kernel an ideal stream-in pattern whose C_{Mem} becomes insignificant. Our model is configured with similar hardware parameters² including: L1/12 MSHR = 12/32, ROB = 112 and $\Delta_{L2M} = 40$. As depicted in Fig. 13b, our model presents good predictions of real-world CPU performance, indicating our modeling is rational and captures the critical factors in the SpMV kernel. Meanwhile, as the experiments match the optimistic model, it is likely the cache bandwidth is not the bottleneck on single-core SpMV, but should be considered in multi-thread cases. The fact that our model is able to match both GEM5 and Intel CPUs with different micro-architectural parameters proves the robustness of our approach.

4.3 SIMD Model

For SIMD model, we define *pipeline efficiency* as the average number of valid operations generated by each issued instruction. As shown in Fig. 14, we consider multiply/add and load/store of SpMV values as valid operations, while control logic or additional pre/post-process as burdens since they don't directly contribute to results. To optimize *pipeline efficiency*, we should pursue lower kernel complexity, higher vector usage, and fewer branch mispredictions.

With better *pipeline efficiency*, in-core performance is optimized. Specifically, $C_{Perfect}$ with SIMD can be written as:

$$C_{Perfect} = \frac{OpNum}{OpDensity \times OpWidth \times IPC^*} \quad (23)$$

$$OpDensity = \frac{OpInsnNum}{InsnNum}, OpWidth = vl \times vr \quad (24)$$

where $OpDensity$ measures the proportion of SIMD arithmetic instructions, and $OpWidth$ is the average number of valid computations performed by one SIMD arithmetic instruction which is determined by vector length vl and average vector usage rate vr . Higher *pipeline efficiency* increases both $OpDensity$ and $OpWidth$ to optimize in-core execution.

Meanwhile, *pipeline efficiency* also influences the memory latency. Consider $L2ReqDensity$ as:

2. Intel Skylake-X CPU micro-architecture has never been officially totally revealed. Therefore we use some assumed parameters that were studied by the community. We use ROB=112 as the hyper-thread technique splits the reorder buffer size.

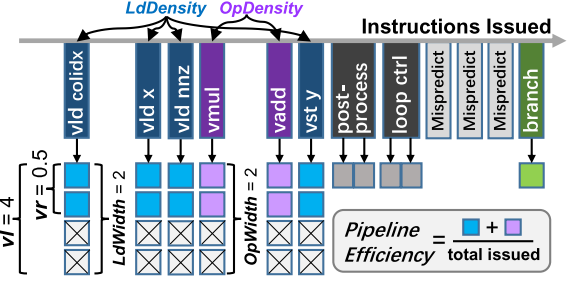


Fig. 14. SIMD model uses the concepts of *pipeline efficiency*, which measure how fast valid operations can be issued to pipeline.

$$L2ReqDensity = L2MissRate \times LdDensity \times LdWidth \quad (25)$$

$$LdDensity = \frac{LdInsnNum}{InsnNum}, LdWidth = vl \times vr \quad (26)$$

where $L2MissRate$ is a stable factor determined by the inherent distribution of a given matrix, $LdDensity$ is the density of SIMD load instructions, and $LdWidth$ is the average number of valid elements in each load. As higher *pipeline efficiency* leads to higher $L2ReqDensity$, it also improves MLP according to Eqs. (16)–(15). But $L2ReqDensity$'s relationship with the impact factor λ is complicated. As in Fig. 12, higher $L2ReqDensity$ is not always beneficial. Therefore, the overall effect of such optimizations should be carefully studied.

4.4 Optimization Factors

Fig. 15 depicts the breakdown time of different matrices. For *scale-free* matrices, memory latency (C_{Mem}) is obviously the major bottleneck, while for *HPC* the CPU pipeline runtime ($C_{pipeline}$) is also critical. While all architecture factors are fixed (e.g., vl , $ROBSize$, Δ_{BP} , Δ_{L2M} , etc), the goal of our optimization is to adjust critical software factors to alleviate the bottlenecks of both matrix types. According to Fig. 12, for *scale-free* matrices increasing $L2ReqDensity(x)$ is usually effective to reduce λ as long as the cache bandwidth is not saturated. This optimization requires to improve $LdDensity$ and $LdWidth$ as derived from Eq. (25). On the other hand, for *HPC* it is also profitable to optimize the pipeline runtime, which can be achieved by improving $OpDensity$ and $OpWidth$ according to Eq. (23). All the above optimizations require increasing the *pipeline efficiency*. Therefore, we propose to avoid pre-/post-process to have lower complexity and increase the essential SIMD instruction density. In addition, avoid using vector masks to guarantee full usage of SIMD vectors and maximize vr . Besides, from Fig. 15 we also learn that branch misprediction penalty C_{BPMiss} is also a critical bottleneck for *HPC*. Therefore we use regular computation patterns to reduce branch mispredictions.

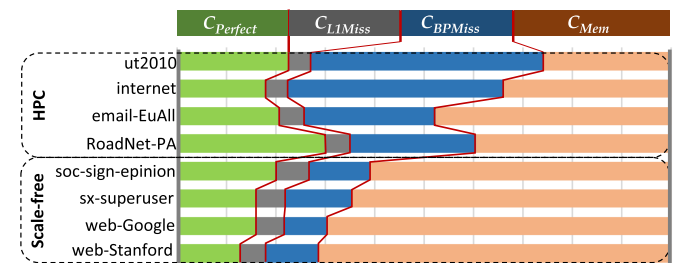


Fig. 15. The breakdown of SpMV execution runtime.

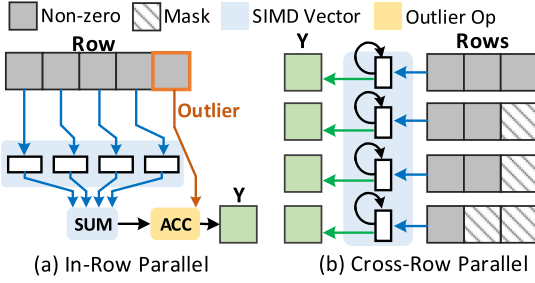


Fig. 16. Example of two basic vectorization strategies: in-row parallelism and cross-row parallelism, on a 4-width SIMD machine.

5 OPTIMIZATION METHOD

Our model reveals the critical factors that have the potential for optimization. In this section, we study how to rationally optimize these factors with an efficient novel SpMV kernel.

5.1 Vectorization Pattern

To pursue low complexity, we first study two basic vectorization patterns in SpMV on SIMD machines: the in-row parallelism and the cross-row parallelism [27], as depicted in Fig. 16. Note that in this paper we do not resort to other more sophisticated patterns as they will inevitably bring extra overheads, as explained in the above sections. In-row parallelism is the most simple SIMD implementation. It collects non-zeros of the same row to multiply and accumulate results in SIMD vectors until the row is over. If the number of values is not the multiple of SIMD width, the outliers are separately calculated, using either scalar operations or masked SIMD operations, as shown in Fig. 16a. Obviously in-row parallel is more efficient on long rows that allow for a higher vector usage rate. However, for sparse matrices where most non-zeros are in short rows, this pattern is likely to suffer from extremely low parallelism. On the other hand, the cross-row parallelism uses SIMD lanes to process a panel of adjacent rows simultaneously, as in Fig. 16b. Rows in the same panel are processed together until the longest row is computed. Vector masks are used to filter out undesired data load and computation in case of uneven rows. In a matrix where adjacent rows have similar lengths, cross-row parallelism has the advantage over in-row parallelism because it can achieve high vector usage even if rows are short. However, as sparse matrices tend to have random and uneven row lengths, one effective optimization for cross-row parallel is to put rows of the same length together so that it is more favorable for cross-row parallelism. To study the two basic vectorization patterns, we compare them on two types of randomly generated sparse structures: matrices with equal-length rows and matrices with random-length rows. As depicted in Fig. 17, cross-row parallelism achieves high performance on equal-length rows but is seriously undermined on random-length rows because many operations are masked-off. It is interesting to observe that for both types the curve of in-row parallelism rises quickly with row length. Even for equal-length rows where cross-row parallelism has the theoretical optimal performance, in-row parallelism can achieve equivalent performance when rows are 40 or longer. This indicates that in-row parallelism is a near-optimal method for long rows.

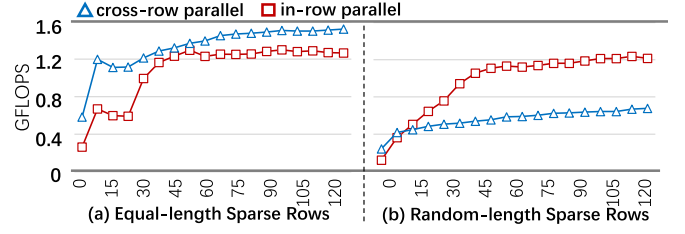


Fig. 17. Single-core performance of in-row parallelism and cross-row parallelism on different sparse structures. X-axis is average row length.

5.2 Kernel Design

5.2.1 Basic Kernel

Our novel kernel, termed as SpV8, aims to achieve near-optimal (>95%) vectorization with extremely low complexity. We discard partial vector operations and vector masks to make sure all SIMD vectors are fully used. Non-zero values that are not included in vectors are computed with simple scalar operations to simplify the computation pattern. Meanwhile, SpV8 avoids using irregular reduction patterns and restricts the final results to be generated in a regular way, so that the amount of conditional branches is minimized. Fig. 18 depicts the pre-processing of the SpV8 kernel in the following steps: *Step-1*: Sort and re-arrange the sparse matrix rows into groups by length, i.e., amount of non-zeros. *Step-2*: For each group, collect rows into panels whose row amount equals the multiple of SIMD width. Left-over rows are termed *fragments*. *Fragments* of all groups are collected as the *fragment stack*. *Step-3*: Transpose all panels into the column-major format for ease of computation. The above steps split a matrix into several panels and one fragment stack. For panels where rows have equal length and are aligned with SIMD width, SpMV applies cross-row parallelism to achieve optimal vectorization, whose computation pattern is very regular. On the other hand, fragment stack consists of uneven rows that are unsuitable for cross-row parallelism. We apply in-row parallelism to compute each fragment row separately since it is proved to be more efficient on such structures (see Fig. 17). The non-vectorized values are computed as scalar operations as shown in Fig. 18. It should be noted that using scalar operations won't reduce the performance as long as the row is long. For example, for rows longer than 40, more than 85% elements are computed in full vectors with high performance. Only a few (<15%) left elements to use scalar operations, which cause little overhead but can significantly simplify the computation since the final results of uneven fragment rows can be generated in sequential order without using vector masks or a dedicated reduction process. As to be explained later, most fragment rows are long enough to guarantee high computation efficiency.

5.2.2 Partitioning

While reordering matrix rows helps create a better vectorization structure for SpV8, it has the side effect of changing the original sequence of the y array. As the y array is originally stream-in data with good locality, permuting its sequence may cause degradation in data locality and bring in extra overheads on memory access, as depicted in Fig. 18.

Moreover, for HPC matrices where most non-zero elements

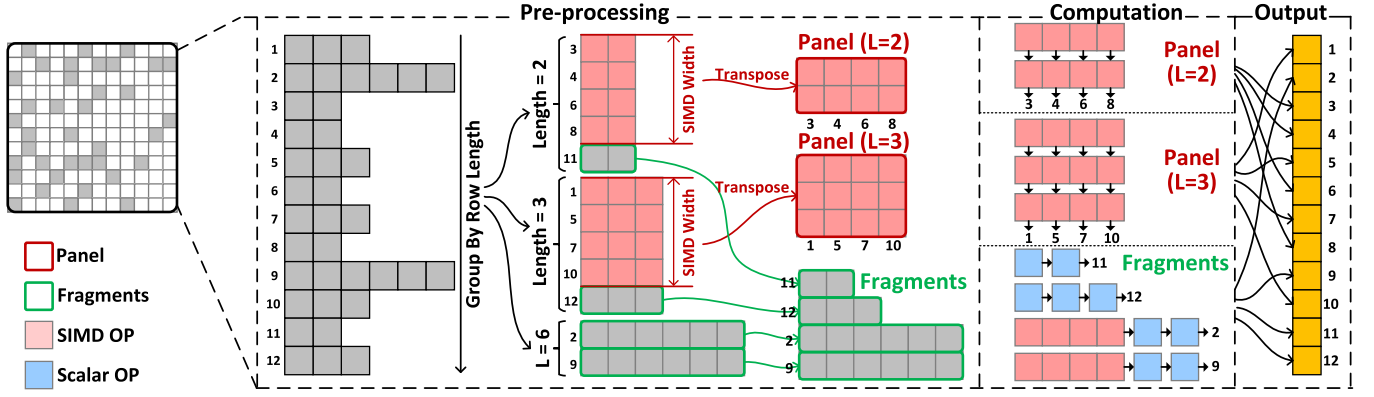


Fig. 18. Example of SpV8 kernel's pre-processing steps and computation flow on a 4-width SIMD machine.

are near the diagonal, disturbing rows will also break the original structured distribution and lead to a worse locality of x data. Therefore, for large matrices, global row reordering is likely to cause significant degradation of memory latency. To solve this problem, we restrict the scale of row reordering by dividing the matrix into multiple partitions and computing each partition separately, so that the memory access permutation is limited. The scale of the partition should be carefully decided. It should be small enough to mitigate the row permutation. But too-small scale will reduce vector usage as it is likely to generate fewer panels and more fragments. Therefore, the choice of partition scale is a trade-off between data locality and vectorization efficiency. To find the optimal scale, we test different partition scales (ranging from 500 to 20 K rows) on two representative matrices: *soc-sign-epinions* as scale-free type, and *ut-2010* as HPC type. The results are in Fig. 19. We first measure the proportion of non-zero values in fragment rows. On both matrices, the proportion of fragments is large when we use small partition scales, taking up more than 10% of total non-zeros. However, the actual non-vectorized (outlier) values are in a much smaller proportion, since many fragment rows are long enough to be well vectorized, as explained in Fig. 17. Therefore, SpV8 achieves a decent vectorization rate ($>90\%$) even with small partition scales, especially for the HPC type. On the other hand, we observe that the partition scale has different influences on two matrices' memory access latency. HPC matrix is more sensitive to scale as its original locality is good and thus more expensive to break. For scale-free matrices that have poor locality, the growth of the partition scale only causes slight increments of memory latency. The above observation leads

to our conclusion that different matrix types should use different partition scales to avoid much memory access penalty while still keeping high vector usage rates. As Fig. 17 shows, we set the partition scale to be 2 K for HPC and 10 K for scale-free to ensure at least 95% non-zeros are vectorized.

5.3 Multi-Thread Optimization

SpV8 can be easily adapted to multi-thread by assigning partitions to threads. To maintain workload balance, partitions should have a similar amount of non-zeros. We define N_p as the threshold amount of non-zeros for partitions:

$$N_p = \frac{nnz}{\max(\frac{RowNum}{R_k}, T \times 4)} \quad (27)$$

where nnz and $RowNum$ are the total amounts of non-zero values and rows in matrix, T is the number of threads, R_k is the row amount of the selected partition scale, which is 2 K or 10 K as described in the above section. This equation guarantees at least $T \times 4$ partitions are created. In order to avoid synchronization or atomic operations, we do not split a row across multiple partitions. To avoid false sharing, we allocate rows in granularity of $\frac{l_{cache}}{l_{data}}$ where l_{cache} is the L1 cache line size (64 Bytes) and l_{data} is the size of each vector y element. When a double floating-point is used, the allocation granularity is 8 rows. Then we create a partition by collecting every 8 rows from the top of the matrix. When the amount of non-zero exceeds the threshold N_p , the current partition is over and we move to the next until all rows are collected. During execution, each partition is converted into SpV8 format and processed by one thread. We use a scheduler to dynamically assign partitions to idle threads to guarantee the workload balance of threads. Moreover, in cases where multiple NUMA nodes are used, we duplicate the read-only CSR data (nnz , $colidx$, $rowptr$) and place a copy to each node at the beginning of the computation. In this way, the threads only need to access local stream-in data so that the advantage of higher DDR bandwidth provided by the NUMA technique can be fully exploited. The x data is not duplicated as we find x is usually not bounded by DDR bandwidth, and the duplication of x would cause extra cost because for each iteration the values of y have to be written to multiple nodes in order to be used in next iteration. Note that this optimization may be infeasible for extremely large matrices, where a performance bottleneck would occur

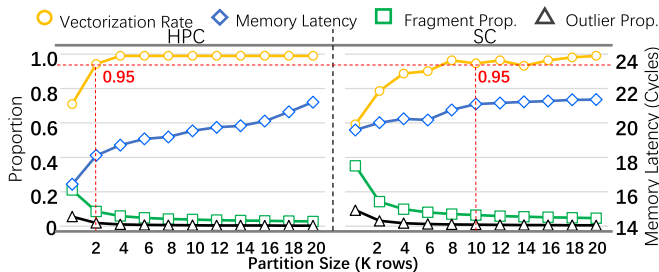


Fig. 19. Influence of different partition scales (ranging from 500 to 20 K) on vectorization and average memory access latency. The HPC matrix is *ut-2010* and scale-free matrix is *soc-sign-epinions*.

TABLE 4
Information of Evaluation Platforms

Architecture	AArch64	x86	x86
Platform	GEM5	Intel Xeon Gold 6146	AMD Ryzen 5950X
Processor	ARMv8 (O3CPU)	Skylake-X	Ryzen 5950X
SIMD	SVE (512-bit)	AVX-512	AVX2
Frequency	2.5 GHz	2.5 GHz	3.4 GHz
L1 D-Cache	32 KB	32 KB	32 KB
L2 Cache	1 MB	1 MB	512 KB
L3 Cache	/	24.75 MB	32 MB

using first-touch data distribution and the matrix data has to be dynamically shared across multiple NUMA nodes.

6 EVALUATIONS

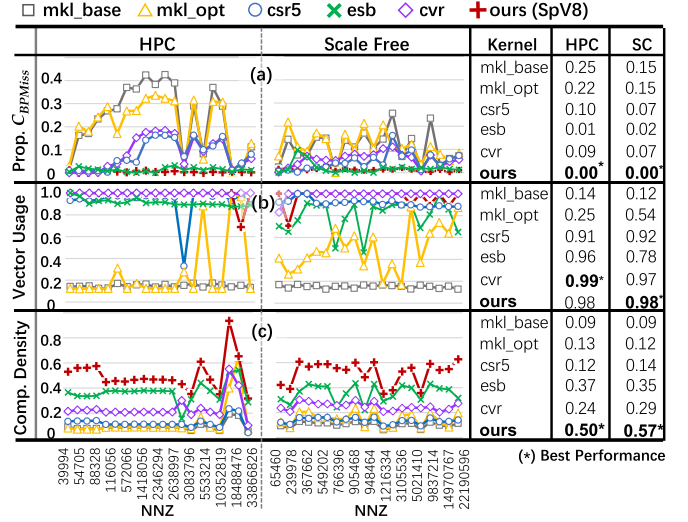
6.1 Experiment Setting

We evaluate SpV8 on commercial x86 platforms of both Intel and AMD CPUs. In order to study the influences of factors such as MKL, MSHR, and ROB, we conduct additional experiments on an AArch64 CPU using a GEM5 simulator. The GEM5 CPU model is O3CPU that can simulate superscalar out-of-order CPUs with high accuracy ($\geq 95\%$) [23]. Table 4 lists the spec of two platforms, where the GEM5 CPU is configured to be similar with x86 features. For both platforms, we set the baseline as official commercial high-performance libraries. For x86, the baseline is Intel Math Kernel Library [10] (MKL version 2020.2). For AArch64, the baseline is ARM Performance Library [28] (APL version 21.1.0), a commercial high-performance library. Moreover, on x86 CPU we compare with several state-of-the-art approaches including ESB [15], CVR [12], CSR5 [11], SELL-C- σ [14] and CSR2 [16]. All approaches are tested on their available open-source code or implemented by ourselves if the original code is not available. We evaluate multi-thread kernel only on x86, as GEM5 takes too long simulation time for multi-core systems. For the benchmark, we use the well-known sparse matrix collection of SuiteSparse [18]. We select total 287 sparse matrices of both HPC and scale-free types various scales up to 2.3 billion non-zeros.

6.2 Factors Optimization

This section shows SpV8's effect on optimizing the target factors listed in Fig. 15. SpV8 and other compared approaches of MKL, ESB, CVR and CSR5 are evaluated on single-thread Intel CPU. All metrics are collected with VTuneProfiler [29], an official profiling tool for x86 CPU. The baseline MKL kernel includes two variants, a naive version (*MKL_base*) with basic SIMD implementation and an advanced one (*MKL_opt*) that has several available optimization methods. *MKL_opt* uses a three-step *Inspect-Optimize-Execute* manner to first analyzes the input matrix shape and then select appropriate optimization methods accordingly.

We first evaluate the branch prediction penalty by measuring the proportion of wasted μ -op pipeline slots caused by bad speculations (i.e., $\frac{C_{BPMiss}}{C_{All}}$). As shown in Fig. 20a, the problem of miss-prediction is critical on MKL kernels as expected. CVR and CSR5 alleviate the problem to some



(*) Best Performance

Fig. 20. Comparison of critical performance factors for multiple kernels on single-core execution. HPC and scale-free matrices are separately tested and the average metric values of each type are listed on the right.

extent but still suffer from notable penalties due to their complex control flows. Meanwhile, SpV8 shows the minimal (near to zero) penalty of all kernels. We can also notice that C_{BPMiss} penalty tends to be more critical in HPC than in scale-free matrices, which is consistent with our performance model. Then we measure the average vector usage rate in Fig. 20b to evaluate factors of *OpWidth* and *LdWidth*. The baseline kernel *MKL_base* has a low usage rate ($< 20\%$) due to the irregular matrix shapes. *MKL_opt* improves the usage of scale-free matrices as it uses in-row parallelism that can process uneven rows better. In contrast, ESB uses cross-row parallelism which is more efficient on HPC matrices where row lengths are less irregular. SpV8, CSR5, and CVR have high usage rates on both matrix types. Finally, Fig. 20c evaluates the effective computation density by measuring the value of $OpDesntiy * OpWidth$. MKL has low density as expected. Meanwhile, though CSR5 and CVR have high vector usage, their density is quite low owing to complex computation patterns, indicating that the advantage of high vector usage can be undermined if the kernel is complex. SpV8's density is higher than all the other kernels with great margins, because SpV8 is designed to pursue minimal complexity to reduce the total instruction amount.

Above results show that SpV8 can greatly optimize the target factors and outperforms other state-of-the-art kernels.

6.3 MLP Optimization

To evaluate *MLP*, we use the GEM5 ARM AArch64 platform to compare SpV8 with the ARM Performance Library (APL). Similar to *MKL_opt*, APL provides two SpMV approaches: a simple SIMD implementation (*APL_base*) and an advanced kernel which follows the *Inspect-Optimize-Execute* scheme like *MKL_opt*. Both kernels are evaluated in our experiments. Considering the simulation costs, we reduce the benchmark to 20 matrices with an NNZ range of 65K–2.4 M.

We first compare kernels with default CPU parameters, i.e., L1/L2 MSHR sizes are 12/32, and ROB size is 112. The results are shown in Fig. 21a. For *APL_base*, *MLP* is quite low ($MLP \approx 3$) on *HPC* matrices because the x data has

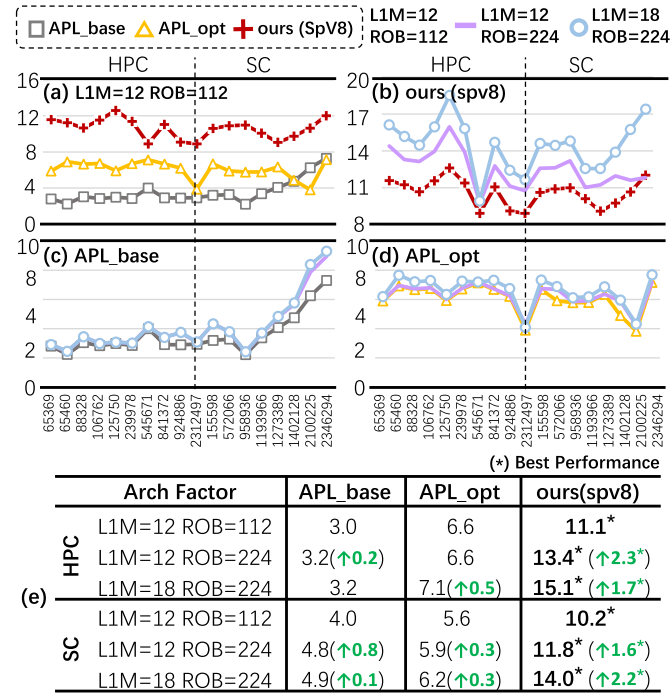


Fig. 21. Comparison of *MLP* for single-core SpV8 and APL kernels on various scales of ROB and L1 MSHR (L1M). The average *MLP* values and changes on different ROB/MSHR settings are also listed.

good locality and rarely triggers L2 miss. On *scale-free* matrices, L2 cache miss rate increases when the kernel randomly accesses x array of larger scale, which causes *APL_base*'s *MLP* to rise, indicating the critical influence of factor $L2ReqDensity(x)$ on *MLP* as suggested in Eq. (18). The *APL_opt* kernel effectively improves the value *MLP* to be 4~7 as it improves the usage of SIMD vectors so that more x data are fetched for each load instruction. However, such *MLP* values are still much less than the L1 MSHR size, which is the upper bound for L1 cache parallelism $P_d(t)$ according to Eq. (14). This indicates that *APL_opt* fails to fully exploit the L1 MSHR bandwidth. Besides, *APL_opt*'s *MLP* becomes equivalent or even worse than *APL_base* on large *scale-free* matrices, which indicate *APL_opt* is inefficient to accelerate large random sparse matrices. Meanwhile, SpV8 achieves much higher *MLP* on both matrix types, while the *MLP* values are close to or even higher than L1 MSHR size, proving the L1 MSHR bandwidth is fully used by SpV8.

We also study restriction effects of architecture factors, i.e., ROB size and MSHR size. Eqs. (18) and (14) explains their influences on *MLP*: ROB size affects maximal number of potential parallel cache misses encountered during out-of-order execution [6], and MSHR size is the upper bound of parallel cache misses. According to Eq. (18), *MLP* can be bounded by ROB if $L2ReqDensity$ is so low that memory requests generated within instruction window can't fill up L1 MSHR. In contrast, if $L2ReqDensity$ is high, *MLP* would be limited by the capacity of L1 MSHR since there are too many parallel requests to handle. In Figs. 21b, 21c, and 21d, we test with enlarged L1 MSHR size (as 18) and ROB size (as 224) to observe the bottleneck of kernels. The changes of *MLP* are listed in Fig. 21e. It is obvious that *APL_base* is limited by ROB, as its *MLP* increases when larger ROB is used.

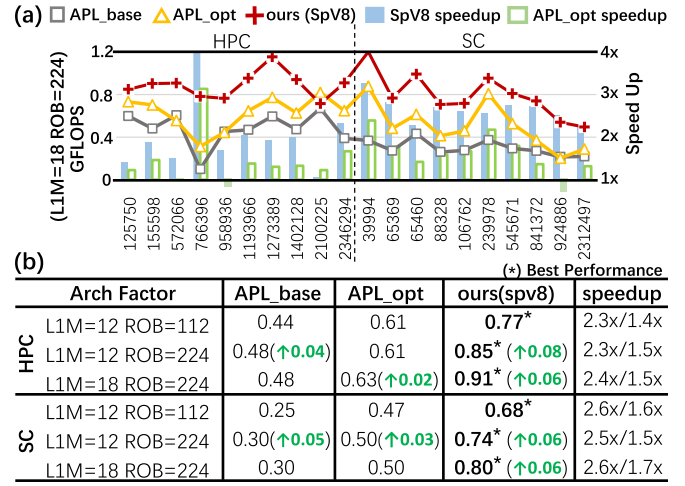


Fig. 22. Single-core performance of SpV8 and APL kernels on GEM5 AArch64 CPU using different L1 MSHR (L1M) and ROB settings. The performance changes caused by different MSHR/ROB sizes are listed.

but is hardly improved with a larger L1 MSHR size, which is expected considering its *MLP* is too low to saturate L1 MSHR bandwidth. Meanwhile, *APL_opt* is only slightly affected, indicating its *MLP* has other intrinsic restrictions besides ROB and L1 MSHR factors, e.g., the dependency of instructions which limits the execution window. However, due to *APL_opt* is not open-sourced, further investigation is impossible. On the other hand, SpV8 shows good scalability as its *MLP* continues to achieve considerable increments on both matrix types whenever ROB or L1 MSHR gets enlarged, indicating SpV8 can always fully exploit all the available parallelism since it is highly optimized for *MLP* by maximizing $L2ReqDensity$ and eliminating the influence of wrong branch predictions, as indicated in Fig. 15b.

Meanwhile, according to Fig. 12, though *APL_opt* optimizes *APL_base* to greatly increase *MLP* from 3~4 to 6~7, it only slightly reduces λ and therefore results in limited performance improvement. SpV8 improves *MLP* to high level (10~15) which causes a significant reduction of λ , which is expected to effectively reduce memory penalty.

6.4 Overall Performance

We compare the overall performance of SpV8 with several state-of-the-art approaches. The speedup of each approach is t_{base}/t_{opt} , where t_{opt} and t_{base} are the runtime of optimization method and baseline kernel, respectively.

We first compare SpV8 and APL kernels on the GEM5 AArch64 platform using 1-thread. The ROB and L1 MSHR are set to be 224 and 18 respectively, which are representative settings for high-end CPUs, e.g., the Intel Skylake-X architecture. As depicted in Fig. 22a, on both HPC and *scale-free* types, SpV8 has the best performance with large margins: 2.3x/2.6x speedup against the baseline *APL_base* kernel, and 1.5x/1.7x speedup against the *APL_opt* kernel. Fig. 22b presents the performance change under different ROB/MSHR scales. We can observe that SpV8 continues to achieve higher performance with larger ROB/MSHR, while *APL_opt* doesn't benefit from the large sizes as it fails to further improve *MLP*, which is consistent with Fig. 21.

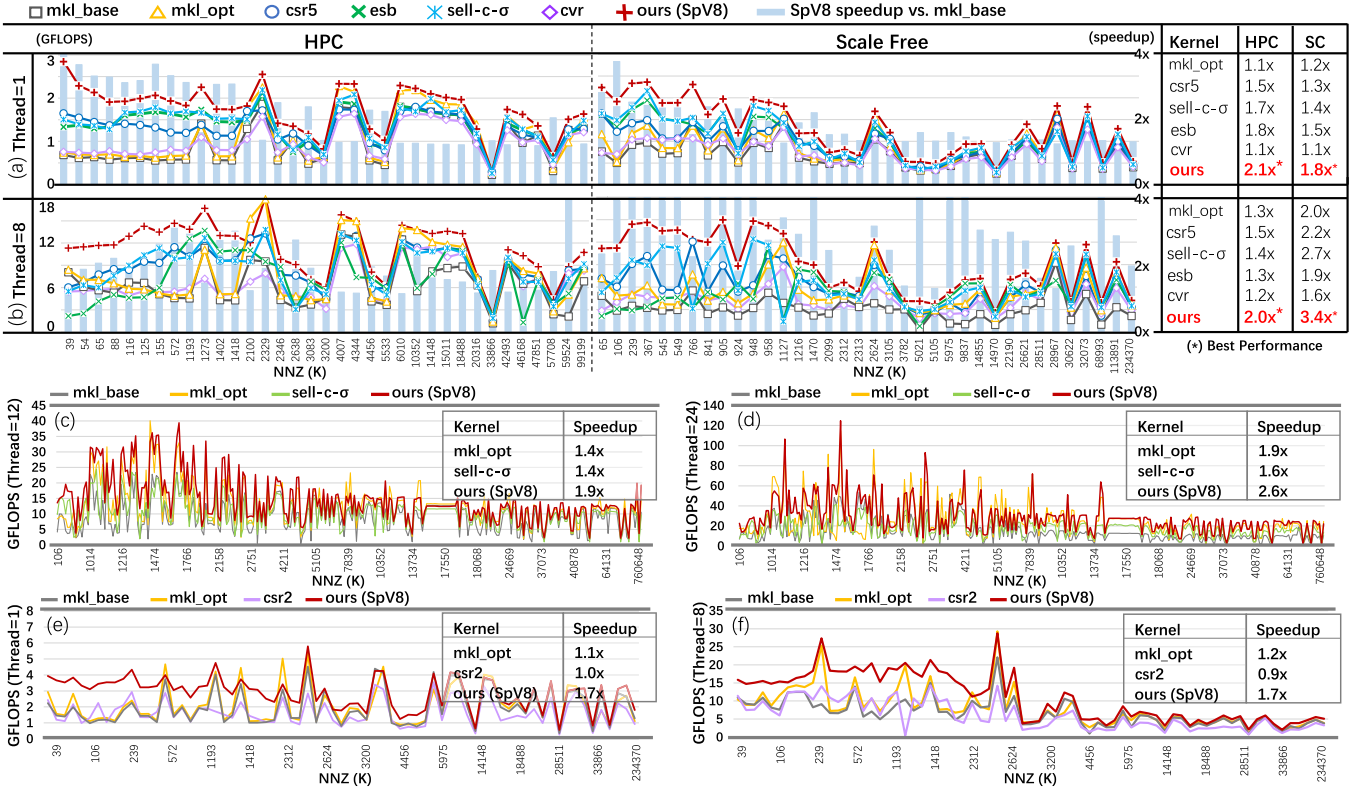


Fig. 23. Performance of several kernels on scale-free and HPC matrices on x86 Skylake-X architecture, using Intel Skylake-X processor with: (a) 1-thread, (b) 8-thread respectively. SpV8's speedup against the baseline is also depicted. More extensive evaluations include: (c)(d) full-socket experiments with one node (12-thread) and two nodes (24-thread); (e)(f) AMD Ryzen CPU using 1 thread and 8 threads, respectively. For each experiment, the average speedup against the baseline of all tested kernels is listed.

Then we evaluate SpV8 on Intel CPU and compare it with other kernels including MKL, CSR5, ESB, CVR, and SELL-C-σ. All kernels are directly from their published repository except SELL-C-σ which is implemented by us.³ Fig. 23a depicts the results of all approaches on HPC and scale-free matrices using 1-thread. Their average speedups against baseline *MKL_base* are also listed. On all tested matrices of both types, SpV8 outperforms other kernels by reaching $2.1\times/1.8\times$ speedup over baseline and $1.3\times/1.4\times$ over the second best kernels, i.e., ESB for HPC and CSR5 for scale-free. We also evaluate multi-thread performance using 8 CPU cores within the same NUMA node on an Intel Xeon server. The results are listed in Fig. 23b, where SpV8 still has the best performance with great margins: $3.4\times/2.0\times$ speedup over baseline. Moreover, to test the performance of full threads, we test all 12 physical cores in one socket (Fig. 23c), and use all 24 physical cores in two sockets (Fig. 23d). Our approach remains the highest average performance in the additional experiments, achieving $1.9\times$ speedup with 12 threads, and $2.6\times$ speedup with 24 threads. It should be noted that multiple-core execution of scale-free matrices is challenging, as it is difficult to achieve a good workload balance. The results on full threads show the scalability of SpV8 on large-scale matrices.

In order to test our kernel on more CPU models, we also run experiments on an AMD Ryzen 5950X processor, which supports AVX2. Our kernel is ported to AMD CPU by using

AVX2 instructions which are 256-bit width. Besides our kernel, we also evaluated MKL and CSR2 [16], which were originally optimized for AVX2. Experiments of 1-thread and 8-thread cases are listed in Figs. 23e and 23f, which show that our kernel achieves the highest performance using AVX2 on the AMD platform. These results further indicate the robustness and generality of our approach. Meanwhile, we can also observe the compared kernel performance gap are relatively smaller than our prior experiments on Intel CPU. This is because AVX2 only provides a 256-bit vector instead of 512-bit in AVX512, which undermines the advantage of SIMD.

The above experiments indicate that SpV8 achieves extensive performance improvements by highly optimizing the CPU pipeline efficiency. This outcome proves that the proposed performance model is insightful for identifying valuable optimization factors, which also shows the necessity of building models to guide optimizations.

6.5 Pre-Processing Overhead

We measure the pre-processing overhead as t_{opt}^{pre}/t_{base} . t_{opt}^{pre} is 1-thread pre-processing runtime. It is normalized by t_{base} , which is 1-thread one-time baseline SpMV kernel execution. As shown in Fig. 24, MKL_opt has the most expensive pre-processing overhead on both HPC and scale-free types, which is on average 13~30 times t_{base} , while CVR has the lightest overheads. Meanwhile, SpV8 has the second lowest overhead which takes only 8 times of baseline runtime. Considering the optimization effect of SpV8 kernel, its pre-

3. To test SELL-C-σ, we set $C = 8$ and $\sigma = 1024$ for all matrices, and use OpenMP dynamic scheduler for multi-thread execution.

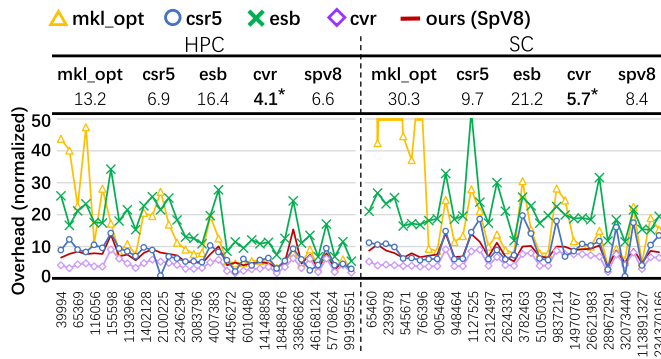


Fig. 24. Pre-processing overhead on all matrices, normalized by the multiple of one-time baseline SpMV execution runtime.

processing cost would be negligible, as it can be easily amortized for the potential hundreds of SpMV iterations.

7 CONCLUSION

In this article, we propose a comprehensive performance model for SpMV's execution on superscalar out-of-order CPUs, which aims to identify the critical factors and reveal their relationships. Based on the insights of our model, we propose SpV8, a new SpMV kernel that is designed to achieve maximal vector usage while having regular computation patterns. Our experiment results on Intel Xeon CPU, AMD Ryzen CPU, and ARM AArch64 CPU show that SpV8 outperforms other state-of-the-art approaches on various matrix types, with very low pre-processing overhead. We believe that the proposed model can provide valuable insights for SpMV optimization research, and SpV8 can be widely applicable in real-world applications. The SpV8 kernel is available at github.com/monkey2000/spv8-public.

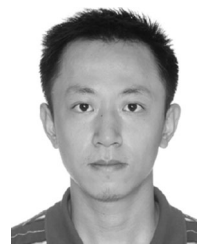
ACKNOWLEDGMENTS

Tian Xia and Gelin Fu contributed equally.

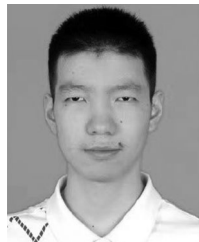
REFERENCES

- [1] A. E. Saryüce, E. Saule, K. Kaya, and Ü. V. Çatalyürek, "Regularizing graph centrality computations," *J. Parallel Distrib. Comput.*, vol. 76, pp. 106–119, 2015.
- [2] P. Zong et al., "PIT: Processing-in-transmission with fine-grained data manipulation networks," *IEEE Trans. Comput.*, vol. 70, no. 6, pp. 877–891, Jun. 2021.
- [3] C. Hong, A. Sukumaran-Rajam, I. Nisa, K. Singh, and P. Sadayappan, "Adaptive sparse tiling for sparse matrix multiplication," in *Proc. 24th Symp. Princ. Pract. Parallel Program.*, 2019, pp. 300–314.
- [4] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, "Speeding up SpMV for power-law graph analytics by enhancing locality & vectorization," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1–15.
- [5] H. Zhao, T. Xia, C. Li, W. Zhao, N. Zheng, and P. Ren, "Exploring better speculation and data locality in sparse matrix-vector multiplication on Intel Xeon," in *Proc. IEEE 38th Int. Conf. Comput. Des.*, 2020, pp. 601–609.
- [6] S. Beamer III, *Understanding and Improving Graph Algorithm Performance*. Berkeley, CA, USA: Univ. California, Berkeley, 2016.
- [7] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [8] H. Stengel, J. Treibig, G. Hager, and G. Wellein, "Quantifying performance bottlenecks of stencil computations using the execution-cache-memory model," in *Proc. 29th ACM Int. Conf. Supercomput.*, 2015, pp. 207–216.

- [9] A. F. Glew, "MLP yes! ILP no," in *Proc. ACM Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 1998.
- [10] E. Wang et al., "Intel Math kernel library," in *High-Performance Computing on the Intel Xeon Phi™*. Berlin, Germany: Springer, 2014, pp. 167–188.
- [11] W. Liu and B. Vinter, "CSR5: An efficient storage format for cross-platform sparse matrix-vector multiplication," in *Proc. 29th Int. Conf. Supercomput.*, 2015, pp. 339–350.
- [12] B. Xie et al., "CVR: Efficient vectorization of SpMV on X86 processors," in *Proc. Int. Symp. Code Gener. Optim.*, 2018, pp. 149–162.
- [13] W. T. Tang et al., "Optimizing and auto-tuning scale-free sparse matrix-vector multiplication on Intel Xeon Phi," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim.*, 2015, pp. 136–145.
- [14] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, "A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide SIMD units," *SIAM J. Sci. Comput.*, vol. 36, no. 5, pp. C401–C423, 2014.
- [15] X. Liu, M. Smelyanskiy, E. Chow, and P. Dubey, "Efficient sparse matrix-vector multiplication on X86-based many-core processors," in *Proc. 27th Int. ACM Conf. Int. Conf. Supercomput.*, 2013, pp. 273–282.
- [16] H. Bian, J. Huang, R. Dong, L. Liu, and X. Wang, "CSR2: A new format for SIMD-accelerated SpMV," in *Proc. IEEE/ACM 20th Int. Symp. Cluster Cloud Internet Comput.*, 2020, pp. 350–359.
- [17] D. Doerfler, "Understanding application data movement characteristics using Intel's VTune amplifier and software development emulator tools," *Intel Xeon Phi Users Group (IXPUG) 2015, Annu. Meeting*, 2015.
- [18] T. A. Davis and Y. Hu, "The university of Florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1–25, 2011.
- [19] A. Yasin, "A top-down method for performance analysis and counters architecture," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2014, pp. 35–44.
- [20] Intel X86-64-IA32 architectures optimization manual, 2022. [Online]. Available: <https://www.intel.com/content/dam/doc/manual/64-ia-32-architectures-optimization-manual.pdf>
- [21] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *Proc. IEEE 13th Int. Symp. High Perform. Comput. Archit.*, 2007, pp. 63–74.
- [22] Y. Chou, B. Fahs, and S. Abraham, "Microarchitecture optimizations for exploiting memory-level parallelism," in *Proc. IEEE 31st Annu. Int. Symp. Comput. Archit.*, 2004, pp. 76–87.
- [23] A. A. Abudaqa, T. M. Al-Kharoubi, M. F. Mudawar, and A. Kobilica, "Simulation of ARM and x86 microprocessors using in-order and out-of-order CPU models with Gem5 simulator," in *Proc. IEEE 5th Int. Conf. Elect. Electron. Eng.*, 2018, pp. 317–322.
- [24] T. Karkhanis and J. Smith, "A first-order superscalar processor model," in *Proc. IEEE 31st Annu. Int. Symp. Comput. Archit.*, 2004, pp. 338–349.
- [25] S. Eyerman, L. Eeckhout, T. Karkhanis, and J. E. Smith, "A mechanistic performance model for superscalar out-of-order processors," *ACM Trans. Comput. Syst.*, vol. 27, no. 2, 2009, Art. no. 3.
- [26] S. Van den Steen and L. Eeckhout, "Modeling superscalar processor memory-level parallelism," *IEEE Comput. Archit. Lett.*, vol. 17, no. 1, pp. 9–12, Jan.–Jun. 2018.
- [27] C. Li, T. Xia, W. Zhao, N. Zheng, and P. Ren, "SpV8: Pursuing optimal vectorization and regular computation pattern in SpMV," in *Proc. IEEE/ACM 58th Des. Automat. Conf.*, 2021, pp. 661–666.
- [28] N. Stephens et al., "The ARM scalable vector extension," *IEEE Micro*, vol. 37, no. 2, pp. 26–39, Mar./Apr. 2017.
- [29] J. Reinders, "An overview of programming for Intel Xeon processors and Intel Xeon Phi coprocessors," *Intel Corporation, Santa Clara*, 2012.



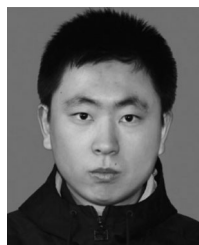
Tian Xia (Member, IEEE) received the PhD degree from the National Institute of Applied Sciences (INSA), France, in 2016. He is an assistant professor with the College of Artificial Intelligence, Xi'an Jiaotong University since 2019. His current research domains include the cloud-computing virtualization, and innovative parallel computation architecture. He is also interested in reinforcement learning algorithms and applications.



Gelin Fu received the bachelor's degree in control science and engineering from Chongqing University, Chongqing, China, in 2019. He is currently working toward the PhD degree with the College of Artificial Intelligence, Xi'an Jiaotong University, Xian, China. His research interests include system modeling and optimization and computer architecture.



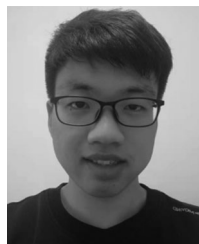
Ruiyang Cheng is currently working toward the undergraduate degree in computer science with Xi'an Jiaotong University, since 2019. His research interests include computer system and high performance computing.



Chenyang Li is currently working toward the undergraduate degree in computer science with Xi'an Jiaotong University, since 2019. His research interests include computer system and high performance computing.



Wenzhe Zhao (Member, IEEE) received the bachelor's, master's, and PhD degrees from Xi'an Jiaotong University, in 2005, 2008, and 2014, respectively. He is an assistant professor with the College of Artificial Intelligence, Xi'an Jiaotong University. During 2011 to 2013, he was a visiting student with Rensselaer Polytechnic Institute (RPI). His current research direction is DNN based computing architecture and VLSI design.



Zhongpei Luo received the BE degree in automation from Xi'an Jiaotong University, in 2021. He is currently working toward the MS degree in pattern recognition and intelligent systems with XJTU. His main research interests include high performance computing, parallel algorithms for dense and sparse algebra, GPGPU technology, hardware security.



Nanning Zheng (Fellow) received the graduate degree from the Department of Electrical Engineering, in 1975, the ME degree in information and control engineering from Xi'an Jiaotong University, in 1981, and the PhD degree in electrical engineering from Keio University, in 1985. He is currently a professor and the director with the Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University. He became a member of the Chinese Academy Engineering in 1999.



Lucheng Zhang is currently working toward the undergraduate degree with the School of Electrical Engineering, Xi'an Jiaotong University, since 2019. He is also a research assistant with PLCT Lab, Institute of Software, Chinese Academy of Sciences since 2022. His research interests include computer architecture, system modeling, and VLSI design.



Pengju Ren (Member, IEEE) received the PhD degree from Xi'an Jiaotong University, in 2012. He is a professor with the College of Artificial Intelligence, Xi'an Jiaotong University since 2015. During 2009 to 2011, he was a visiting PhD student with Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology (MIT). His current research interests include on-chip network and computer architecture.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.